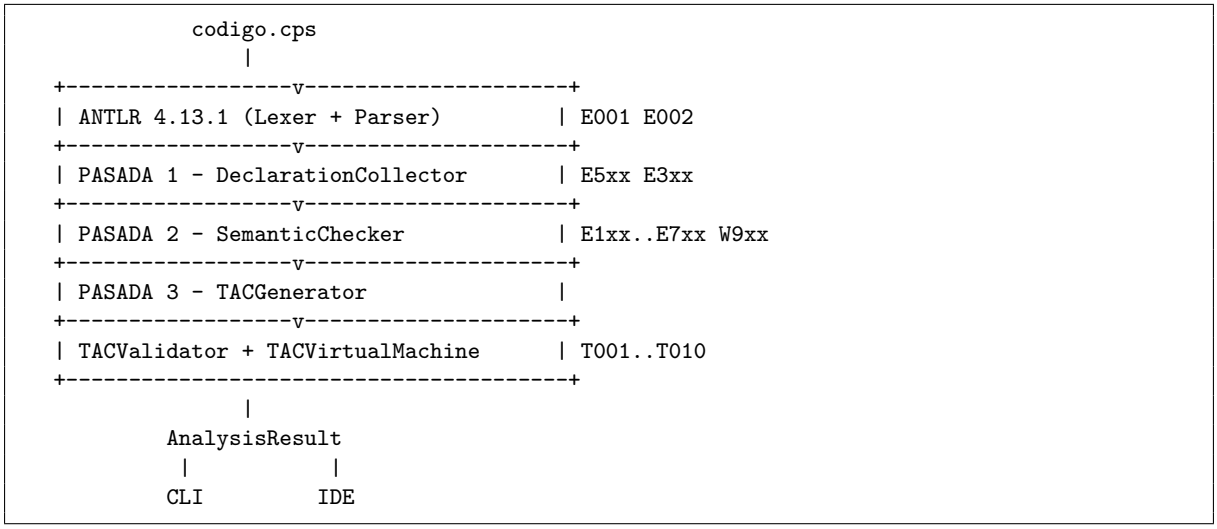
Arquitectura del compilador de Compiscript

Resumen del diseño del compilador: fases implementadas (análisis **léxico y sintáctico** con ANTLR, análisis **semántico** con un visitor propio y generación de **código intermedio** TAC), decisiones tomadas y supuestos sobre el lenguaje.

1. Visión general

El front-end convierte un archivo `.cps` en una **lista de diagnósticos** y una **tabla de símbolos**, que luego consume la generación de código intermedio.



El punto de entrada único es `compiscript.analyze(source)`. El CLI, el IDE y los tests llaman a esa misma función, así que los tres se comportan igual.

2. Módulos principales

Módulo	Responsabilidad
`generated/`	Lexer y parser de ANTLR (no se editan a mano).
`syntax.py`	Puente con ANTLR; traduce los errores al español.
`diagnostics.py`	Catálogo de las 53 reglas y el `ErrorReporter`.
`types.py`	Jerarquía de tipos y reglas de compatibilidad.
`symbols.py`, `scope.py`	Símbolos, ámbitos y asignación de memoria.
`collector.py`	Pasada 1.
`checker.py`	Pasada 2 (reglas semánticas).
`tac/`	Cuádruplas, temporales, generador, validador y máquina virtual.
`analysis.py`, `cli.py`	Orquestador y línea de comandos.

3. Visitor en lugar de Listener

Se eligió **Visitor** para el chequeo porque cada nodo de expresión debe **devolver su tipo** al padre; un Listener no retorna valores y obligaría a mantener una pila paralela. La pasada 1, en cambio, es un recorrido plano que sólo registra declaraciones.

4. Dos pasadas y *hoisting*

Este programa es válido, pero `factorial` se usa antes de declararse:

```
print(factorial(5));
function factorial(n: integer): integer { ... }
```

Antes de visitar cada lista de sentencias se ejecuta la pasada 1, que registra, en orden: (1) nombres de clases, (2) herencia y detección de ciclos (`E505`), (3) miembros en orden topológico y (4) firmas de funciones. El *hoisting* es **por ámbito**: una función anidada no se ve desde fuera. La pasada 2 vuelve a entrar en los ámbitos ya creados en lugar de crear nuevos.

5. Sistema de tipos

```
Type
+-- PrimitiveType  integer, float, boolean, string, null, void
+-- ArrayType(T)   T[]  (invariante en T)
+-- ClassType      instancia (covariante: subclase -> superclase)
+-- FunctionType   (T1, T2) -> R
+-- ErrorType      centinela
```

Sin cascadas de errores. Cuando una expresión falla devuelve `ErrorType`, compatible con todo, y además el `ErrorReporter` deduplica por (código, línea, columna). Así un solo error real produce un solo diagnóstico.

Asignación. `float` acepta `integer` (ensanchamiento), pero no al revés; `null` sólo se asigna a tipos por referencia; una superclase acepta una subclase, pero no al revés; y los arreglos son invariantes (si `float[]` aceptara `integer[]`, se corromperían los enteros originales).

Operadores.

Operador	Regla	Error
`+`	numérico, o concatenación si un lado es `string`	`E101`
`- * /`	numéricos; resultado `float` si alguno lo es	`E101`
`%`	ambos `integer`	`E115`
`&& || !`	`boolean`	`E102`/`E109`
`== !=`	tipos comparables	`E103`
`< <= > >=`	numéricos o `string`	`E104`

6. Tabla de símbolos y entornos

Se crea un `Scope` por programa (`GLOBAL`), función o método (`FUNCTION`), clase (`CLASS`) y bloque (`BLOCK`). La resolución sube por `parent`, lo que implementa anidamiento, sombreado y captura de closures. El cuerpo de una función comparte el ámbito de sus parámetros, de modo que redeclarar un parámetro es `E202`.

Cada símbolo guarda, además de nombre y tipo, lo que necesitarán las fases siguientes: `storage`, `offset`, `size` (`integer` 4, `float` 8, `boolean` 1, referencias 4), `label`, `captured`, tamaños de marco y layout de instancia con `vtable`.

Closures. Al resolver un nombre que vive en una función exterior se marca `captured` y se registra en las `captures` de la función actual **y de todas las intermedias**. Globales, funciones y clases no se capturan.

7. Código intermedio (tercera pasada)

El generador **no vuelve a resolver nombres ni a inferir tipos**: lee las anotaciones que deja la pasada 2 (`types`, `lvalues`, `symbols`, `owners`). Duplicar esa lógica sería una fuente permanente de bugs. La tabla de símbolos suma `ActivationRecord`, `temp_count`, `vtable_slots` y `vtable_label`, y todos los objetos empiezan con una cabecera de 4 bytes con el puntero a su tabla de métodos.

8. Análisis de flujo

- `_terminator`: detecta código muerto (`W902`).
- `_always_returns`: verifica que toda ruta devuelva valor (`E308`); contempla `if/else`, `do-while`, `try/catch` y `switch` con `default`.

`break` y `continue` usan contadores (`loop_depth`, `switch_depth`) que se reinician al entrar en una función.

9. Decisiones de diseño y supuestos

- Se añade `float` a la gramática (cambio retrocompatible).
- `break` es válido dentro de un `switch`; `continue` exige bucle.
- La condición del `switch` **no** se exige `boolean`; en su lugar cada `case` debe ser comparable con el sujeto (`E405`). `if`, `while`, `do-while` y `for` sí exigen `boolean`.
- `string + valor` es concatenación (lo usa el ejemplo del curso).
- La variable de `catch` es un `string`.
- Un índice constante fuera de rango es *advertencia*, no error.
- Dentro de un método se puede usar un atributo sin `this.`
- El constructor no se somete a la comprobación de sobrescritura (`E508`).
- Una función sin `: T` es un procedimiento (`void`).
- Con errores de sintaxis no se ejecuta el análisis semántico.
- Los parámetros requieren anotación de tipo (`E310`).
- `W905` (variable sin usar) sólo aplica a ámbitos no globales.

10. Extensión a la gramática

Único cambio respecto a la gramática original:

```
baseType: 'boolean' | 'integer' | 'float' | 'string' | Identifier;
Literal: FloatLiteral | IntegerLiteral | StringLiteral;
FloatLiteral: [0-9]+ '.' [0-9]+;
```

`FloatLiteral` va antes que `IntegerLiteral` para que `3.14` sea un solo token (*maximal munch*). El programa de ejemplo del curso sigue analizándose sin errores.

11. El IDE

Un backend Flask delgado (`/api/analizar`) llama a `compiscript.analyze` y devuelve JSON al editor Monaco, que muestra problemas, árbol sintáctico, tabla de símbolos y tokens. Monaco está incluido localmente para funcionar sin internet.

El panel del árbol tiene dos ejes: **detalle** (compacto o completo) y **vista** (indentada o gráfica en SVG). El modo compacto colapsa la cascada de precedencia y reduce el árbol del programa de ejemplo de 1758 a 774 nodos (56 %), conservando los tipos inferidos.

12. Estrategia de pruebas

377 tests en dos niveles: (1) unitarios, con al menos un caso exitoso y uno fallido por cada una de las 53 reglas; (2) programas completos, 7 válidos y 7 inválidos con el diagnóstico esperado anotado línea por línea (`// @error E101`). Se exige exactamente lo anotado y nada más, lo que detecta también **falsos positivos**.